

Random roast generator in C++

"Write a C++ console program that asks the user for their name, stores at least 10 roast templates in a `string` array or `vector<string>`, randomly selects one using `rand()` and `srand(time(0))`, replaces the `{name}` placeholder with the actual input using `string::find()` and `string::replace()`, then prints the final roast."

Created a file, ran a command >

Compiles and runs perfectly. Here's the file:

Presented file >

Here's a breakdown of every requirement fulfilled:

Data storage — 14 roast templates live in a `vector<string>`, each containing the `{name}` placeholder (sometimes at the start, middle, or end to keep things varied).

Randomization — `srand(time(0))` seeds the RNG once at startup, and `rand() % roasts.size()` picks a random index on each run.

Placeholder replacement — Uses `string::find()` and `string::replace()` inside a `while` loop so every occurrence of `{name}` in a template is swapped out, not just the first one.

Edge case — If the user hits Enter without typing anything, the name defaults to `"Mystery Human"` so the roast still makes sense.

To compile and run:

```
bash
g++ -o roast_generator roast_generator.c
./roast_generator
```

Write a message...

+

Sonnet 4.6



Claude is AI and can make mistakes. Please double-check responses.



- + New chat
- Projects
- Artifacts
- Customize
- Code Upgrade

Pinned

Drag to pin

Recents

- Random roast generator in C++
- Random roast generator project
- Impact Lab web application
- Bank account management system
- C++ gravity simulator project
- Code in preferred language
- Infrared security system project
- Chocolate consumption game
- Minimum window size for bubble
- Counting coprime factor problem
- Minimum range length to turn
- Counting coprime factor problem
- Physics simulation for bubble

Mahi Arora · Full

Random roast generator in C++

Replace the vector initialization block with this:

cpp

```
std::vector<std::string> roasts;
roasts.push_back("Hey {name}, I'd roast you, but my mom said I'm not allowed to burn t
roasts.push_back("{name}, you're the reason they put instructions on shampoo bottles."
roasts.push_back("If brains were petrol, {name} wouldn't have enough to power a flea's
roasts.push_back("{name}, you're not stupid - you just have bad luck thinking.");
roasts.push_back("I'd agree with you, {name}, but then we'd both be wrong.");
roasts.push_back("{name}, you have your entire life to be an idiot. Why not take today
roasts.push_back("Some drink from the fountain of knowledge, {name}. You just gargled.
roasts.push_back("{name}, you're like a cloud - when you disappear, it's a beautiful d
roasts.push_back("I'm not saying {name} is dumb, but they once tried to alphabetize M&
roasts.push_back("{name}, cancel your internet - you are living proof it's wasted on s
roasts.push_back("I'd explain it to you, {name}, but I left my crayons at home.");
roasts.push_back("{name}, you're proof that even evolution takes a day off sometimes."
roasts.push_back("Brains aren't everything, {name}. In your case, they're nothing.");
```

Option 1 is the better long-term habit — `-std=c++11` (or `-std=c++17`) should really always be passed when writing modern C++. Most tutorials and codebases assume it.

Add error handling to my C++ roast generator: (1) If the user enters an empty name or only spaces, print 'Error: Name cannot be empty. Please try again.' and loop until valid input is given. (2) If the name contains digits or special characters, print 'Error: Name must contain letters only.' and re-prompt. (3) Wrap the main logic in a try-catch block for unexpected errors."

9:53 PM



Ran 2 commands, viewed a file, created a file >

Write a message...



Sonnet 4.6



Claude is AI and can make mistakes. Please double-check responses.

🏠 🔍

+ New chat

📁 Projects

🔗 Artifacts

⚙️ Customize

📄 Code Upgrade

Pinned

📌 Drag to pin

Recents

Random roast generator in C++

Random roast generator project

Impact Lab web application

Bank account management system

C++ gravity simulator project

Code in preferred language

Infrared security system project

Chocolate consumption game

Minimum window size for bubble

Counting coprime factor pairs

Minimum range length to turn

Counting coprime factor pairs

Physics simulation for bubble

Random roast generator in C++

Ran 2 commands, viewed a file, created a file >

All three error paths work perfectly. Here's the updated file:

Presented file >

Here's exactly what was added and why each decision was made:

Two helper functions (above `main`)

- `isBlank(s)` — loops through every character with `isspace()`. Returns `true` if nothing non-whitespace is found. This correctly catches `" "`, `" "`, `"\t"`, etc.
- `isLettersOnly(s)` — uses `isalpha()` and allows spaces so first+last names like `"John Doe"` pass. Anything else (digits, `@`, `!`, `-`) triggers the error.

Both cast characters to `unsigned char` before passing to `isspace` / `isalpha` — this is required by the C standard to avoid undefined behaviour on platforms where `char` is signed.

Input validation loop (`while (true)`)

Enter name → blank? → "Error: Name cannot be empty..." → re-prompt
→ non-letters? → "Error: Name must contain letters only." → re-prompt
→ valid → break

The loop never exits until clean input arrives, so no further code ever sees a bad name.

try-catch block wrapping all of `main`

Get base class of exception (the base class of unexpected exception). Two places see this.

Write a message...

+ Sonnet 4.6 🔊

Claude is AI and can make mistakes. Please double-check responses.



- + New chat
- Projects
- Artifacts
- Customize
- Code Upgrade

Pinned

Drag to pin

Recents

- Random roast generator in C++
- Random roast generator project
- Impact Lab web application
- Bank account management system
- C++ gravity simulator project
- Code in preferred language
- Infrared security system project
- Chocolate consumption game
- Minimum window size for bubble
- Counting coprime factor problem
- Minimum range length to turn
- Counting coprime factor problem
- Physics simulation for bubble

Mahi Arora · Full

Random roast generator in C++



Compile command (unchanged from before):

```
bash
g++ -std=c++11 -o roast roast_generator.cpp
./roast
```



Roast generator
C++

Open in Visual Studio Code

After displaying a roast in my C++ program, ask the user 'Want another roast? (y/n):'. If they enter y, generate a new roast making sure it's different from the previous one by tracking the last index used. If they enter n, print a funny goodbye message. If they enter anything else, show 'Error: Invalid choice. Enter y or n only.' and re-prompt."

Ran 2 commands, viewed a file

Everything passes. Three different roasts for Alice, no repeats, bad inputs correctly rejected.

Presented file

Here's a breakdown of every change made:

`pickNewIndex(lastIndex, total)` — a dedicated helper that loops `rand() % total` until it gets an index different from the previous one. The `lastIndex` starts at `-1` (guaranteed to

Write a message...

+

Sonnet 4.6



Claude is AI and can make mistakes. Please double-check responses.



Random roast generator in C++

Ran 2 commands, viewed a file

Everything passes. Three different roasts for Alice, no repeats, bad inputs correctly rejected.

Presented file

Here's a breakdown of every change made:

`pickNewIndex(lastIndex, total)` — a dedicated helper that loops `rand() % total` until it gets an index different from the previous one. The `lastIndex` starts at `-1` (guaranteed to match nothing), so the very first roast is also random with no special-casing needed. The edge case where only one roast exists is handled by returning `0` immediately to avoid an infinite loop.

`applyName(roast, name)` — the placeholder-replacement logic was pulled out of `main` into its own function so the roast loop can call it cleanly on each iteration without duplicating code.

Two nested loops replace the old single-roast flow:

```
outer loop → pick roast → print it
    inner loop → prompt "y/n"
        y → break inner (outer fires again with new roast)
        n → print goodbye → return 0
    else → "Error: Invalid choice. Enter y or n only." →
re-prompt
```

This structure means `y / Y` and `n / N` are both accepted (case-insensitive), anything else — including blank input, spaces, or multi-character strings — hits the error message and loops back without advancing.

Compile (same as before):

Write a message...

+

Sonnet 4.6



Claude is AI and can make mistakes. Please double-check responses.

The screenshot displays the Claude AI web interface. On the left is a sidebar with navigation options: 'New chat', 'Projects', 'Artifacts', 'Customize', and 'Code' (which is selected and has an 'Upgrade' link next to it). Below these are 'Pinned' and 'Recent' sections. The 'Recent' section lists several projects, with 'Random roast generator in C++' at the top, highlighted in blue. The main area shows the details of this project. At the top, it says 'Random roast generator in C++' with a dropdown arrow. Below this is a code editor showing a 'bash' terminal session with the commands: `g++ -std=c++11 -o roast roast_generator.cpp` and `./roast`. Under the code editor is a file icon and the text 'Roast generator C++'. To the right of this is a button that says 'Open in Visual Studio Code'. Below the code editor is a large text box containing a prompt: 'Extend the C++ roast generator to support 3 categories: coding, laziness, and general. Use a map<string, vector<string>> to store templates per category. Ask the user to choose a category, validate the input, and if invalid print 'Error: Invalid category. Choose coding, laziness, or general.' then re-prompt. Pick a random roast from the selected category.' Below the prompt is a section titled 'Ran 2 commands >' with the text: 'All paths work — invalid category rejected, "Coding" normalised to "coding", no repeated roasts, bad y/n re-prompted.' Below this is a section titled 'Presented file >' with the text: 'Here's a breakdown of every addition:'. Below the text is a code editor showing the C++ code: `map<string, vector<string>> structure`. Below the code editor is a text input field with the placeholder 'Write a message...'. To the right of the input field is a dropdown menu showing 'Sonnet 4.6' and a microphone icon. At the bottom of the interface is a footer that says 'Claude is AI and can make mistakes. Please double-check responses.'

Random roast generator in C++

Each key is the category name; the value is its own pool of 10 roasts. Using `map` means validation is free — `categories.find(category) != categories.end()` tells you in one call whether the input is valid, and adding a fourth category later requires zero changes to the validation logic.

Note the space in `>>` — required in C++03/C++11 to avoid the `>>` token being parsed as the stream operator. Modern compilers with `-std=c++11` and later handle it fine either way, but the space is the safe habit.

`toLowerCase()` helper

```
cpp
category = toLower(category);
```

Applied immediately after reading input, so `"Coding"`, `"CODING"`, and `"coding"` all resolve to the same map key without needing three separate checks.

Category validation loop (mirrors the existing name loop)

```
blank or not in map → "Error: Invalid category. Choose coding, laziness, or
general." → re-prompt
valid key → break
```

`vector<string>& pool` — a reference, not a copy

```
cpp
```

Write a message...

+

Sonnet 4.6



Claude is AI and can make mistakes. Please double-check responses.

Random roast generator in C++

`vector<string>& pool` — a reference, not a copy

cpp

```
std::vector<std::string>& pool = categories[category];
```

The `&` means the roast loop works directly on the map's data with no copying. `lastIndex` is still tracked per-session so you never see the same roast twice in a row within a category.

Compile (unchanged):

bash

```
g++ -std=c++11 -o roast roast_generator.cpp  
./roast
```



Roast generator
CPP

[Open in Visual Studio Code](#)

Polish my C++ roast generator: (1) Add a welcome banner using `cout` with `=` separators and a title. (2) Use `this_thread::sleep_for(chrono::seconds(1))` from `<thread>` and `<chrono>` for a dramatic pause before showing the roast. (3) At the end of the session, display how many roasts were generated using a counter variable. Make the program feel complete and fun.



10:02 PM

Write a message...



Sonnet 4.6



Claude is AI and can make mistakes. Please double-check responses.



- + New chat
- Projects
- Artifacts
- Customize
- Code Upgrade

Pinned

Drag to pin

Recents

- Random roast generator in C++
- Random roast generator project
- Impact Lab web application
- Bank account management system
- C++ gravity simulator project
- Code in preferred language
- Infrared security system project
- Chocolate consumption game
- Minimum window size for bubble
- Counting coprime factor problem
- Minimum range length to turn
- Counting coprime factor problem
- Physics simulation for bubble

Mahi Arora · Full

Random roast generator in C++

```
std::cout << "   Preparing your roast";  
for (int i = 0; i < 3; ++i) {  
    std::this_thread::sleep_for(std::chrono::milliseconds(600));  
    std::cout << " .";  
    std::cout.flush();           // force each dot to appear immediately  
}  
std::this_thread::sleep_for(std::chrono::seconds(1));
```

`std::cout.flush()` after each dot is essential — without it, the terminal buffers the output and all three dots appear at once when the loop ends, killing the effect. The roast itself is shown only after the full pause, so the anticipation lands.

Roast counter and session summary

`int roastCount = 0` is incremented with `++roastCount` every time a roast is displayed. When the user types `n`, it feeds directly into the goodbye banner:

```
=====
                        SESSION COMPLETE
=====
                        Victim : Alice
                        Category : coding
                        Roasts delivered: 2
```

Compile command (add `-lpthread` on Linux if you get a linker error for `<thread>`):

```
bash  
g++ -std=c++11 -o roast roast_generator.cpp -lpthread  
./roast
```

Write a message...

+

Sonnet 4.6



Claude is AI and can make mistakes. Please double-check responses.

- + New chat
- Projects
- Artifacts
- Customize
- Code Upgrade

Pinned
Drag to pin

- Recents
- Random roast generator in C++
 - Random roast generator project
 - Impact Lab web application
 - Bank account management system
 - C++ gravity simulator project
 - Code in preferred language
 - Infrared security system project
 - Chocolate consumption game
 - Minimum window size for bubble
 - Counting coprime factor problem
 - Minimum range length to turn
 - Counting coprime factor problem
 - Physics simulation for bubble

Mahi Arora · Full-time

Random roast generator in C++

`int roastCount = 0` is incremented with `++roastCount` every time a roast is displayed. When the user types `n`, it feeds directly into the goodbye banner:

```
=====
                SESSION COMPLETE
=====
                Victim : Alice
                Category : coding
                Roasts delivered: 2
```

Compile command (add `-lpthread` on Linux if you get a linker error for `<thread>`):

```
bash
g++ -std=c++11 -o roast roast_generator.cpp -lpthread
./roast
```

On macOS, `-lpthread` isn't needed.



Roast generator
CPP

Open in Visual Studio Code

📄 🌟 💬 🔄



Write a message...

+

Sonnet 4.6



Claude is AI and can make mistakes. Please double-check responses.

